AGENDA CLARA

Sistema de gestión de clientes, servicios y reservas

ASIGNATURA Herramientas de Desarrollo	GRUPO 05
ENTREGA Informe de avance adaptado a la consigna	REPOSITORIO github.com/Eduardo-Jara03/Proyecto-pedidos-tiendatecnologia-grupo05

INTEGRANTES REGISTRADOS

- Eduardo Jesús Jara Moya — U21302190
- Lisbert Wiliams Galvan Zamudio — U22217814
- Nery Emerson Velasquez Pacheco — U23272063
- Carlos Fernando Rivera Hipolito — U23216154
- Bruno Santiago Reyes Vicente — U19221035

Documento actualizado para reflejar el caso de gestión de clientes, servicios y reservas. Confirmar integrantes, evidencias y estado real de la colaboración antes de la entrega.

1. Descripción de la solución

Agenda Clara es una aplicación web para centralizar la gestión de clientes, servicios y reservas de una empresa de servicios. La interfaz consume una API REST construida con Node.js y Express, y SQLite almacena los datos con relaciones entre clientes, servicios y reservas.

2. Requerimientos de la primera versión

ID	Requerimiento	Estado del código preparado
RF01–RF03	Registrar, modificar y consultar clientes	Implementado; probar en Windows
RF04–RF06	Registrar/consultar servicios; precio y duración	Implementado; pruebas API
RF07–RF10	Registrar, consultar y cancelar reservas; cliente, servicio, fecha/hora	Implementado; pruebas API

Para versiones posteriores quedan: recordatorios automáticos, reportes, dashboard de indicadores, pagos en línea e historial ampliado de clientes.

3. Arquitectura técnica

Frontend web responsivo; API REST en Express; persistencia SQLite. Las rutas principales son /api/clientes, /api/servicios y /api/reservas. El endpoint /api/health permite verificar la API y la conexión a la base de datos.

4. Actividades y trazabilidad

La matriz docs/matriz-trazabilidad.md relaciona RF01–RF10 con actividad, archivo o endpoint. El archivo docs/tablero-actividades.csv es una plantilla importable para organizar actividades. Los responsables reales, issues, ramas, PR, revisores y commits de integración deben completarse con datos verificables del equipo.

5. Estrategia de ramas e integración

Se propone main para la versión estable, develop para integración, ramas feature/clientes, feature/servicios y feature/reservas para funcionalidades, y docs/entrega-academica para documentación. Los cambios deben integrarse mediante Pull Requests revisadas por alguien distinto del autor. La estrategia y el procedimiento se detallan en docs/estrategia-ramas.md.

6. Pruebas

En el entorno de preparación se ejecutó npm test: 20 pruebas aprobadas y 0 fallidas para API y flujos de clientes, servicios y reservas. El equipo debe repetir las pruebas en Windows después de copiar los archivos y hacer una prueba manual en navegador; guardar la salida como evidencia propia.

7. Conflicto de integración

La consigna requiere al menos un conflicto real provocado por trabajo simultáneo. docs/conflicto-git-plantilla.md ofrece un formato de registro y un ejercicio reproducible, pero no constituye evidencia de un conflicto ya ocurrido. Completarlo después de realizar el ejercicio y adjuntar la captura y el commit reales.

8. Gestión de versiones

La V1 prevista contiene los tres módulos y sus pruebas. Antes de declarar una versión estable, el equipo debe verificar manualmente los flujos, completar la revisión de PR, integrar cambios y crear una etiqueta real (por ejemplo v1.0.0). Consultar docs/versionado-v1.md.

9. Evidencias pendientes para la entrega final

☐ Tablero GitHub Projects accesible con los cinco estados y actividades asignadas.

☐ Enlaces a ramas y Pull Requests publicadas; evidencia de revisión real por un compañero.

☐ Registro de contribuciones y responsables verificados.

☐ Conflicto real resuelto y documentado con captura/commit.

☐ Tag y commit de versión estable.

☐ Captura de ejecución en Windows y validación manual en navegador.

☐ Verificación de asistencia y sustentación obligatoria.

10. Conclusiones

La adaptación técnica alinea el producto con el caso de clientes, servicios y reservas. El cumplimiento completo también depende de ejecutar el código en el equipo y aportar evidencias auténticas de tablero, colaboración, revisión, conflicto y liberación. No se debe afirmar que esos hitos están completados hasta verificarlos en GitHub.